Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Иудин Е.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 30.10.26

Москва, 2026

Постановка задачи

Вариант 8.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и

взаимодействие между ними в одной из двух операционных систем. В результате работы

программа (основной процесс) должен создать для решение задачи один или несколько дочерних

процессов. Взаимодействие между процессами осуществляется через системные сигналы/события

и/или через отображаемые файлы (memory-mapped files).

Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Вариант задания остается с лабораторной N1.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); – создает дочерний процесс.
- ssize_t read(int fd, void *buf, size_t count); – чтение данных из файлового дескриптора.
- ssize_t write(int fd, const void *buf, size_t count); – запись данных в файловый дескриптор.
- void _exit(int status); – немедленное завершение текущего процесса с кодом status.
- int close(int fd); – закрывает открытый файловый дескриптор.
- pid_t wait(int *status); – заставляет родительский процесс приостановиться и ждать, пока не завершится любой из дочерних процессов.
- int dup2(int oldfd, int newfd); – дублирует файловый дескриптор oldfd под новым newfd.
- int open(const char *pathname, int flags, mode_t mode); – создает или открывает файл по пути pathname и возвращает его файловый дескриптор.
- int execl(const char *path, const char *arg0, ..., (char *)NULL); – ОС полностью стирает текущую программу, выполняющуюся в текущем процессе, и запускает новую по pathname.
- int shm_open(const char *name, int oflag, mode_t mode); – создает или открывает по имени объект разделяемой памяти и возвращает его файловый дескриптор.
- int ftruncate(int fd, off_t length); – устанавливает размер файла (в данном случае объекта разделяемой памяти) равным length байт.
- void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset); – отображает файл или объект разделяемой памяти в адресное пространство процесса и возвращает указатель на начало отображения.
- int munmap(void *addr, size_t length); – удаляет отображение памяти из адресного пространства процесса.
- int shm_unlink(const char *name); – удаляет объект разделяемой памяти с указанным именем.

- sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value); – создает или открывает именованный семафор и возвращает указатель на него.
- int sem_wait(sem_t *sem); – уменьшает значение семафора на единицу; если значение равно нулю, процесс блокируется, пока другой процесс не выполнит sem_post.
- int sem_post(sem_t *sem); – увеличивает значение семафора на единицу и пробуждает процесс, ожидающий на нем.
- int sem_close(sem_t *sem); – закрывает семафор в текущем процессе.
- int sem_unlink(const char *name); – удаляет именованный семафор из системы.
- pid_t getpid(void); – возвращает идентификатор текущего процесса.
- pid_t getppid(void); – возвращает идентификатор родительского процесса.

Структура самой программы мало поменялась с первой лабораторной работы. Прежде всего было убрано межпроцессное взаимодействие с использованием pipe. В parent.c открываем shared memory (open_shm) с именем, которое зависит от pid (getpid), затем меняем ее размер с помощью ftruncate. Далее привязываем эту shared memory с помошью mmap. Затем создаем семафор, имя которого также зависит от pid (sem_open). Делаем fork, а потом уже читаем данные дочернего процесса из shared memory, при этом не забывая про синхронизацию с помощью семафора. В child.c происходят практически аналогичные действия, за исключением ftruncate. Также для того чтобы использовать один и тот же семафор и shared memory нам понадобится системный вызов getppid, который позволяет получить pid родительского процесса. Реализуем метод send_msg, который будет отправлять байты родительскому процессу, также не забываем про семафор. Также будем считать, что UINT32_MAX означает завершение передачи информации.

Код программы

parent.c

```
#include <unistd.h>

#include <fcntl.h>

#include <stdio.h>

#include <stdint.h>

#include <sys/wait.h>

#include <sys/mman.h>

#include <semaphore.h>


#define SHM_SIZE 4096


void write_string(int fd, char *s) {
```

```
    size_t len = 0;
    while (s[len] != '\0') len++;
    write(fd, s, len);
}

int main() {
    int status;
    char filename[256];

    write_string(STDOUT_FILENO, "enter file's name (max 255 symbols): ");

    size_t b_read = 0;
    char ch;

    while(b_read < sizeof(filename) - 1) {
        int res = read(STDIN_FILENO, &ch, 1);
        if (res <= 0 || ch == '\n' || ch == '\r') break;
        filename[b_read++] = ch;
    }

    filename[b_read] = '\0';

    if (b_read == 0) {
        write_string(STDOUT_FILENO, "empty file");
        _exit(1);
    }

    int file_fd = open(filename, O_RDONLY);
    if (file_fd < 0) {
        write_string(STDOUT_FILENO, "can't open the file");
```

```
    _exit(1);
  }

  char shm_name[64], sem_name[64];
  snprintf(shm_name, sizeof(shm_name), "/lab-shm-%d", getpid());
  snprintf(sem_name, sizeof(sem_name), "/lab-sem-%d", getpid());

  int shm = shm_open(shm_name, O_RDWR | O_CREAT | O_EXCL, 0600);
  if (shm == -1) {
    write_string(STDOUT_FILENO, "can't open shared memory");
    perror(shm_name);
    _exit(1);
  }

  if (ftruncate(shm, SHM_SIZE) == -1) {
    write_string(STDOUT_FILENO, "can't resize shared memory");
    _exit(1);
  }

  char *shm_buf = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED,
shm, 0);
  if (shm_buf == MAP_FAILED) {
    write_string(STDOUT_FILENO, "can't map shared memory");
    _exit(1);
  }

  sem_t *sem = sem_open(sem_name, O_RDWR | O_CREAT | O_EXCL, 0600, 1);
  if (sem == SEM_FAILED) {
    write_string(STDOUT_FILENO, "failed to create semaphore");
    _exit(1);
```

```
}

pid_t pid = fork();

if (pid < 0) {
    write_string(STDOUT_FILENO, "failed to fork");
    _exit(1);
}

if (pid == 0) {
    dup2(file_fd, STDIN_FILENO);
    close(file_fd);

    execl("./child", "child", NULL);

    write_string(STDOUT_FILENO, "failed exec");
    _exit(1);
}

else {
    close(file_fd);

    uint32_t *length = (uint32_t *)shm_buf;
    char *text = shm_buf + sizeof(uint32_t);

    int running = 1;
    while (running) {
        sem_wait(sem);

        if (*length == UINT32_MAX) {
```

```
            running = 0;
        } else if (*length > 0) {
            write(STDOUT_FILENO, text, *length);
            *length = 0;
        }

        sem_post(sem);
    }

    wait(&status);

    sem_close(sem);
    sem_unlink(sem_name);
    munmap(shm_buf, SHM_SIZE);
    close(shm);
    shm_unlink(shm_name);
  }

  _exit(0);
}
```

child.c

```
#include <unistd.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <semaphore.h>
```

```
#define BUF_SIZE 2048
#define CHUNK_SIZE 1024
#define SHM_SIZE 4096

char *shm_buf;
sem_t *sem;

void send_msg(const char *data, uint32_t len) {
    uint32_t *length = (uint32_t *)shm_buf;
    char *text = shm_buf + sizeof(uint32_t);

    while (1) {
        sem_wait(sem);
        if (*length == 0) {

            if (len != UINT32_MAX) {
                memcpy(text, data, len);
            }

            *length = len;
            sem_post(sem);
            return;
        }
        sem_post(sem);
    }
}

int main(void) {
    char shm_name[64], sem_name[64];
```

```
    snprintf(shm_name, sizeof(shm_name), "/lab-shm-%d", getppid());
    snprintf(sem_name, sizeof(sem_name), "/lab-sem-%d", getppid());

    int shm = shm_open(shm_name, O_RDWR, 0);
    if (shm == -1) {
        const char msg[] = "error: failed to open SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        _exit(1);
    }

    shm_buf = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm,
0);
    if (shm_buf == MAP_FAILED) {
        const char msg[] = "error: failed to map SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        _exit(1);
    }

    sem = sem_open(sem_name, O_RDWR);
    if (sem == SEM_FAILED) {
        const char msg[] = "error: failed to open semaphore\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        _exit(1);
    }

    char buf[BUF_SIZE];
    int head = 0, tail = 0, bytes = 0;
    bool eof = false, last = false, num_ends = false;
    int num = 0, res = 0;
    int sign = 1, cnt = 0;
```

```
while (!last) {
    if (!eof && BUF_SIZE - bytes >= CHUNK_SIZE) {
        int n = BUF_SIZE - tail;
        if (n > CHUNK_SIZE) n = CHUNK_SIZE;
        int r = read(STDIN_FILENO, buf + tail, n);
        if (r <= 0) {
            eof = true;
        } else {
            tail = (tail + r) % BUF_SIZE;
            bytes += r;
        }
    }

    char c;
    if (bytes > 0) {
        c = buf[head];
        head = (head + 1) % BUF_SIZE;
        bytes--;
    } else if (eof) {
        c = '\n';
        last = true;
    } else {
        continue;
    }

    if (c >= '0' && c <= '9') {
        num = num * 10 + (c - '0');
        num_ends = true;
        continue;
```

```
    }

    if (num_ends) {
        if (cnt > 0 && num == 0) {
            const char msg[] = "error: dividing by zero\n";
            send_msg(msg, sizeof(msg) - 1);
            send_msg(NULL, UINT32_MAX);
            _exit(1);
        }
        res = cnt > 0 ? res / num * sign : num * sign;
        cnt++;
        num = 0;
        num_ends = false;
        sign = 1;
    }

    if (c == '-') sign = -1;

    if (c == '\n') {
        if (cnt > 0) {
            char out[32];
            int len = snprintf(out, sizeof(out), "%d\n", res);
            send_msg(out, len);
        }
        cnt = 0;
        res = 0;
        sign = 1;
    }
}
```

```
    send_msg(NULL, UINT32_MAX);

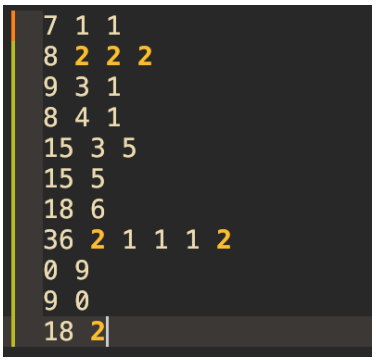
    _exit(0);

}
```

Протокол работы программы

Для работы программы, необходимо предварительно скомпилировать parent.c и child.c, причем child.c с именем child.

Для следующего входного файла:



Результатом запуска parent.c будет:

```
4oddy@MacBook-Pro-Egor-3 lab1 % gcc -o child child.c && gcc -o parent parent.c && ./parent
Введите имя файла (max 255 символов): file.txt
7
1
3
2
1
3
3
9
0
Ошибка: деление на ноль
```

Вывод

В ходе выполнения лабораторной работы я изучил новый метод межпроцессного взаимодействия (IPC), а также научился организовывать его через разделяемую память и семафор. Передача данных, которая была организована в первой лабораторной работе была кардинально перестроена: раньше данные шли через pipe, а на данный момент используется более быстрый вариант через разделяемую память.